SAMSUNG INNOVATION CAMPUS · SIC 8 · BIG DATA ENGINEERING TRACK

PROJECT 03 · SUPPLY CHAIN & INTELLIGENT LOGISTICS

Enterprise Fleet Telematics Lakehouse & Dynamic Logistics Warehouse

Full project plan: architecture, data strategy, model, KPIs, tooling, team split and 6-day execution schedule

Stack	Kafka + Airbyte → Amazon S3 (bronze) → Snowflake (Snowpipe, PySpark on Snowpark Connect, dbt, Iceberg tables) → FastAPI → React + three.js, orchestrated by Airflow
Delivery model	Hybrid: local Docker (generator, Kafka, Airbyte, Airflow, API, web app) + cloud (S3, Snowflake)
Team	3 engineers: Member A (Data & Ingestion), Member B (Snowflake, PySpark & dbt), Member C (Platform, API & Dashboard)
Duration	6 build days (Day 1–6), demo after Day 6
Inputs	Kaggle "Logistics Operations Database" (DS1) and "Logistics and Supply Chain Dataset" (DS2), plus a custom 5,000-vehicle telemetry generator
Document	Plan v2.0 · 22 Oct 2026 · prepared under senior data-engineering review

1 Executive summary

We will build a working telematics data platform for a simulated fleet of **5,000 refrigerated and dry delivery vehicles** on fixed logistics corridors. Kaggle data supplies the master data (trucks, drivers, routes, facilities, loads, fuel history) and calibrates realistic behaviour; a Python generator produces live telemetry that references those same IDs. Events stream through **Kafka** and batch files through **Airbyte** into an immutable **S3** bronze zone. **Snowpipe** loads them into **Snowflake**, a **PySpark** job (Snowpark Connect) cleans them into silver **Iceberg** tables, and **dbt** builds the gold star schema and KPI marts. A **FastAPI** backend serves the KPIs to a **React + three.js** dashboard with a live 3D fleet map. **Airflow** orchestrates every step.

Stakeholders & use case

Fleet operations manager (fuel cost per route), cold-chain quality lead (temperature compliance), safety coach (driver idling and harsh driving). They need one consistent, queryable view instead of three disconnected streams.

Hybrid split

Everything that produces, moves, orchestrates and presents data runs locally in Docker. Storage and heavy compute run in the cloud: S3 for the lake, Snowflake for loading, processing, warehousing and SQL serving.

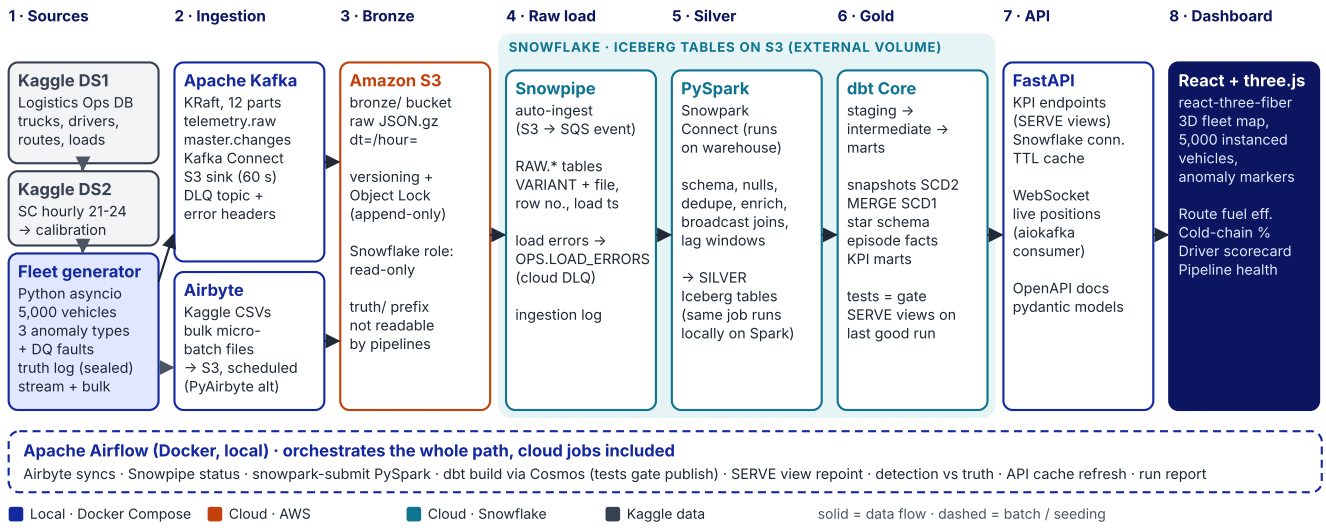
What makes it gradeable

Every anomaly the generator injects is written to a sealed truth log. We can then show that detection rules find what was injected (precision and recall), without the rules ever reading the labels.

Measurable objectives (what "done" means)

#	Objective	Target	Evidence
O1	Simulate the required fleet on fixed corridors	5,000 distinct `vehicle_id` in silver for one run	Snowflake count query + generator config
O2	All telemetry fields present	GPS lat/lon, OBD-II fuel rate, RPM, engine temp, reefer temp: ≥ 99% non-null outside injected gaps	dbt test results
O3	Nothing silently lost	Kafka offsets = S3 records = RAW + load errors; RAW = silver + quarantine (exact)	Reconciliation model, ingestion log
O4	No duplicates in curated data	0 duplicate (vehicle_id, event_ts, seq_no) in silver	dbt `unique_combination_of_columns`
O5	Anomaly detection matches injected truth	Recall ≥ 95%, precision ≥ 90% per family	Detection-vs-truth report
O6	KPIs are trustworthy	Dashboard = API = SQL = hand calculation on a sampled vehicle (±0.1%)	KPI reconciliation sheet
O7	Freshness	Kafka → S3 ≤ 1 min; S3 → RAW ≤ 2 min; event → gold KPI ≤ 15 min	Health page, COPY_HISTORY
O8	Reproducible	Fresh clone + `.env` → `make up && make demo` on a second laptop in ≤ 30 min	Recorded run + README
O9	Measured, not assumed, performance and cost	Events/s, load latency, job runtimes, Snowflake credits per run	Benchmark CSV, query profiles, metering views

2 Solution architecture



End-to-end data flow

1. **Seed:** Airbyte lands DS1 and DS2 unchanged in `s3://bronze/kaggle/`; Snowpipe loads them into `RAW`; dbt builds the initial dimensions. DS2 is profiled once to produce `config/calibration.json` (hour-of-day traffic factors, fuel-rate and temperature distributions, driver-behaviour profiles).
2. **Generate:** the generator expands DS1 trucks and drivers to 5,000 vehicles, assigns them DS1 routes and loads, and emits one ping per vehicle at a configurable interval. *Stream* mode publishes to Kafka; *bulk* mode writes hourly micro-batch files that Airbyte lands on a schedule.
3. **Land:** the Kafka Connect S3 sink writes every record as received to `s3://bronze/telemetry/dt=/hour=/` (JSON.gz, rotated every 60 s). Bronze is append-only and protected by versioning and Object Lock.
4. **Load:** Snowpipe auto-ingests each new file (S3 event → SQS) into `RAW.TELEMETRY` as VARIANT with file name, row number and load time. Files or rows that fail to load are captured in `OPS.LOAD_ERRORS`.
5. **Clean (silver):** a PySpark job running on Snowpark Connect enforces the schema, handles nulls, deduplicates, enriches with dimensions (broadcast joins) and computes per-ping features with window functions. It writes `SILVER` Iceberg tables.
6. **Model (gold):** dbt builds snapshots (SCD2), dimensions, transaction/episode/snapshot facts and KPI marts as Iceberg tables. dbt tests are the quality gate: `SERVE` views are repointed only after a fully green run.
7. **Serve:** FastAPI queries the `SERVE` views (cached), and also streams live vehicle positions from Kafka over WebSocket. The React app renders KPI pages and a three.js 3D fleet map.
8. **Operate:** Airflow runs the chain, retries with backoff, and writes a run report (counts, test results, timings, credits) to `OPS.PIPELINE_RUNS`.

Senior review: the new stack against the handbook

Handbook requirement	Status	How this stack meets it
Pillar 2: Kafka or managed equivalent; periodic landing; immutable raw on S3	Met	Kafka for streams, Airbyte for scheduled landing (NiFi-equivalent role), S3 with Object Lock as bronze.
Pillar 3: PySpark batch jobs or Structured Streaming (**required**)	Met by design	Snowflake + dbt alone would fail this pillar. The silver job is written in PySpark and runs on Snowflake through Snowpark Connect; the same script runs on local Spark for tests and tuning evidence.
Pillar 4: Iceberg or Delta with ACID, time travel, MERGE, pruning, SCD1/2, star schema	Met	Snowflake-managed Iceberg tables on S3 (external volume); dbt incremental MERGE; dbt snapshots for SCD2; Snowflake time travel (AT / BEFORE); clustering + pruning shown in query profiles.

Handbook requirement	Status	How this stack meets it
Pillar 4: SQL engine (Trino, Presto, Hive or cloud OLAP)	Met	Snowflake is listed by the handbook as a cloud option.
Pillar 4: pre-commit quality gates	Met	dbt tests (+ dbt_expectations) run after each model; consumers read only `SERVE` views pinned to the last fully tested run (write-audit-publish).
Pillar 5: Apache Airflow DAGs (**required**)	Kept	Snowflake Tasks would not satisfy this. Airflow drives Airbyte, Snowpipe checks, PySpark, dbt (Cosmos) and the API cache.
Pillar 5: Superset, Power BI, Metabase or *equivalent* dashboards	Met	Custom React + three.js app backed by FastAPI is an equivalent, provided every KPI comes from curated gold data (the live map is an extra).
Local and cloud both	Hybrid by tier	Local: generator, Kafka, Airbyte, Airflow, API, web app (+ local Spark test run). Cloud: S3 + Snowflake. There is no local Snowflake, so the warehouse tier is cloud-only; this is stated as a design decision in the deck.

3 Data strategy: how the Kaggle data and the generator fit together

> **Day 1 verification required.** The file and column lists below are what we expect from the two Kaggle pages. Member A confirms the real schemas in the Day 1 profiling report, and the data contract is adjusted before generator work starts. Anything renamed is handled in dbt staging models, not in downstream jobs.

Source	Expected content	Role in our platform	Lands in
DS1 Logistics Operations Database (trucking ops 2022–2024)	trucks, trailers, drivers, customers, facilities, routes, loads, trips, fuel purchases, maintenance records, delivery events, safety incidents	**Master and reference data.** Seeds vehicle/driver profiles, corridors (origin–destination pairs), cargo types and facility geofences. Fuel purchases give a real benchmark to sanity-check simulated km/L.	Airbyte → bronze/kaggle → RAW.DS1_* → dim_vehicle, dim_driver, dim_route, dim_facility, dim_customer, dim_cargo, fact_fuel_purchase
DS2 Logistics & Supply Chain dataset (hourly, 2021–2024)	timestamped GPS, fuel consumption rate, IoT temperature, traffic congestion, driver behaviour/fatigue scores, delay and risk indicators	**Calibration and history.** Fits distributions the generator samples from, so simulated behaviour follows real patterns. Also kept as a historical context fact.	Airbyte → bronze/kaggle → RAW.DS2 → calibration.json; fact_network_conditions_hourly
Generator (ours)	live pings for 5,000 vehicles + master-data change events + sealed anomaly truth log	**The streaming workload** required by Pillar 1 and Project 03.	Kafka → bronze/telemetry → RAW → SILVER → GOLD

Linking the generator to the real data

- **Fleet expansion to 5,000:** vehicle `VH-00001…VH-05000` maps deterministically to DS1 truck `i mod N` and inherits make, model year, fuel profile and reefer capability. `source_truck_id` is kept for lineage, so every simulated vehicle traces back to a real record. Drivers are expanded the same way.
- **Fixed corridors:** each distinct DS1 origin–destination route becomes a corridor. City coordinates come from a free city lookup (e.g. SimpleMaps basic CSV); a polyline is built by interpolating waypoints every ~1 km. This is not road-accurate, and that is a documented limitation. The same corridor file feeds the three.js map, so the 3D lines match the data exactly.
- **Trips:** DS1 loads define cargo type (cold or dry), reefer setpoint, customer and destination facility. The generator replays loads as trips along their corridor.
- **Realism from DS2:** speed follows hour-of-day congestion factors, fuel rate follows the fitted distribution scaled by speed and load, reefer noise follows the IoT temperature variance, and drivers get a behaviour profile (calm / average / aggressive) from DS2 score quantiles.

- **Units:** if DS1 is imperial (miles, gallons), dbt staging converts to km and litres using factors in a seed file. Gold is metric only.

Telemetry event contract (JSON, schema v1, validated with JSON Schema)

Field	Type / unit	Notes
`event_id`	UUID	Unique per emission; resends keep the same id.
`vehicle_id, driver_id, trip_id, route_id`	string	Foreign keys to DS1-derived masters.
`seq_no`	long	Monotonic per vehicle; with `event_ts` forms the dedupe key.
`event_ts` / `sent_ts`	ISO-8601 UTC	Simulated event time vs wall-clock send time (kept separate on purpose).
`lat, lon`	deg (WGS84)	GPS position.
`speed_kmh, max_decel_mps2`	km/h, m/s²	Peak deceleration since previous ping (for harsh-braking events).
`rpm, engine_temp_c, fuel_rate_lph, fuel_level_pct, odometer_km`	OBD-II	Engine and fuel signals.
`ignition_on, reefer_on`	bool	Needed for idling and cold-chain logic.
`reefer_temp_c, reefer_setpoint_c`	°C	Null when no reefer; null burst = sensor gap.
`schema_version, source`	string	`v1` / `v2`; `stream` or `bulk`.

Other topics: `fleet.master.changes` (driver reassignment, depot moves, reefer swaps, route corrections, used to demonstrate SCD) and `fleet.telemetry.dlq`. A v2 schema adds `tire_pressure_kpa` on Day 4 to demonstrate Iceberg schema evolution (`ALTER ICEBERG TABLE … ADD COLUMN`).

Injected anomalies vs independent detection rules

Family	How the generator injects it (written to sealed truth log)	How the pipeline detects it (never reads labels)
Cold-chain breach	Reefer drift or door-open event: temperature rises 4–10 °C above setpoint for 15–90 min on cold loads. Rate configurable per vehicle-day.	Cold load and `reefer_temp_c > setpoint + 3 °C` for ≥ 15 consecutive minutes → one excursion episode (gaps-and-islands in dbt).
Excessive idling	Vehicle stops away from facilities with engine running (RPM 550–900) for 12–60 min.	`ignition_on`, speed < 2 km/h, RPM 500–1000 for ≥ 10 min, and > 300 m from any DS1 facility geofence.
Route deviation	Vehicle leaves its corridor by 3–15 km for 10–40 min, then rejoins.	Distance to assigned corridor polyline > 2 km for ≥ 3 consecutive pings (computed in the PySpark silver job with broadcast corridor segments).
Data-quality faults	Malformed JSON, missing keys, out-of-range values, duplicates, late events (up to 30 min), sensor null bursts.	Kafka Connect DLQ / Snowpipe load errors; quarantine table; dedupe; incremental MERGE for late data; null flags preserved.

Independence rule: Member A writes injection, Member B writes detection from the written specification only. The truth log lives under `s3://bronze/truth/`, which is excluded from the Snowflake storage integration used by pipelines; only the validation harness (separate role) can read it.

Volume and run profiles (event time is separate from wall-clock time)

Profile	Vehicles	Ping interval	Simulated span	Events	Use
`smoke`	200	60 s	1 h	~12 k	CI and unit tests (local Spark)

Profile	Vehicles	Ping interval	Simulated span	Events	Use
`standard`	5,000	60 s	6 h (compressed ×6 → ~1 h wall)	~1.8 M	Daily integration, dashboards
`full`	5,000	30 s	24 h (compressed ×12 → ~2 h wall)	~14.4 M	Benchmark and final evidence

Full profile ≈ 2,000 events/s at ~350 bytes each (~5 GB raw JSON before gzip). Achieved throughput and Snowflake credits are measured and reported, not assumed.

4 Engineering design by pillar

Pillar 1 · Domain-specific data generation (owner: A)

- Python 3.11 **asyncio**: vehicles are sharded across tasks (e.g. 50 shards × 100 vehicles), and each shard runs a time-stepped loop with an async Kafka producer (`aiokafka`).
- Faker generates driver names, VIN-like IDs and plates; numpy samples the DS2-calibrated distributions.
- `config/generator.yaml` holds seed, profile, anomaly rates and durations, sinks and time compression. The same seed and config give the same output, which satisfies reproducibility.
- **Bulk mode** writes hourly NDJSON micro-batches to `landing/`; Airbyte syncs them to bronze on a schedule. This covers "scheduled bulk micro-batches". A fixed CSV is never used as the only input.

Pillar 2 · Ingestion (owner: A)

- **Kafka (KRaft, Docker)**: `fleet.telemetry.raw` with 12 partitions keyed by `vehicle_id` (keeps per-vehicle order), `acks=all`, idempotent producer, lz4, 7-day retention.
- **Kafka Connect S3 sink**: JSON.gz, time-based partitioner `dt=/hour=`, rotation every 60 s or 10,000 records. Native dead-letter queue: `errors.tolerance=all`, `errors.deadletterqueue.topic.name=fleet.telemetry.dlq` with error context headers.
- **Retry with backoff**: producer uses exponential backoff with jitter (0.2 s → 6.4 s, 6 tries); after that the record goes to the DLQ topic and a local spill file, so nothing is lost. Connect uses `errors.retry.timeout` and `errors.retry.delay.max.ms`.
- **Airbyte**: connections for Kaggle files (File / S3 source → S3 destination) and generator bulk files, triggered by Airflow. Airbyte OSS runs on the demo machine; on 16 GB dev laptops use **PyAirbyte** inside an Airflow task (same connectors, far less RAM).
- **Immutable bronze on S3**: writers only PUT new objects; versioning + Object Lock (governance mode, short retention) on the bronze bucket; Snowflake's storage integration has read-only access.
- **Snowpipe** (auto-ingest via S3 event notifications) loads bronze into `RAW.*` tables (VARIANT + `METADATA$FILENAME`, `METADATA$FILE_ROW_NUMBER`, load time). Failures are pulled from `COPY_HISTORY` / `VALIDATE_PIPE_LOAD` into `OPS.LOAD_ERRORS`, the cloud-side DLQ.
- **Ingestion log**: `OPS.INGESTION_LOG` combines Connect offsets, Airbyte sync stats and Snowpipe COPY_HISTORY per batch (records in/ok/failed, min/max event time, latency). `dlq_replay.py` re-submits fixed records, which proves failures are recoverable.

Pillar 3 · Distributed processing with PySpark (owner: B)

- **Runtime**: `silver_telemetry.py` is a PySpark DataFrame job. In production it runs on a Snowflake warehouse through **Snowpark Connect for Apache Spark**, submitted by Airflow with `snowpark-submit`. The same script runs on a local Spark container against the smoke profile for unit tests (pytest + chispa) and for Spark tuning evidence.
- **Schema enforcement**: explicit `StructType` from the contract; VARIANT parsed with that schema; records that do not conform go to `SILVER.QUARANTINE` with a reason.
- **Nulls**: key fields null → quarantine. Sensor nulls are kept and flagged (`reefer_signal_lost`, `gps_gap`), never imputed silently.
- **Dedupe**: composite business key (`vehicle_id, event_ts, seq_no`) using `row_number()` over a window ordered by load time; incremental by load-time watermark, with MERGE so late events land correctly.
- **Enrichment**: **broadcast** joins to current vehicle, driver, route and corridor dimensions (all small).

- **Windows**: `lag()` per vehicle for distance and fuel since previous ping; distance to corridor; rolling 15-min reefer average. No repeated full scans.
- **Shuffle control**: repartition by `vehicle_id` before windows, AQE and tuned `spark.sql.shuffle.partitions` on the local run; no `groupByKey`. On Snowpark Connect, query profiles show the join and pruning plan.
- **Storage**: output is Snowflake-managed Iceberg tables (Parquet data files on S3), clustered by `(event_date, vehicle_id)`. The local run writes Snappy Parquet partitioned by `dt` to show physical partitioning explicitly.

Pillar 4 · Lakehouse & dimensional warehouse with Snowflake + dbt (owner: B)

- **Iceberg tables in Snowflake** (`CATALOG='SNOWFLAKE'`, external volume on `s3://lakehouse/`) for silver and gold: ACID writes, schema evolution, MERGE, time travel (`AT` / `BEFORE`, 7-day retention) and pruning via file statistics. Snowflake handles compaction for managed Iceberg tables.
- **dbt Core (dbt-snowflake)** layers: `staging` (typed views over silver and DS1/DS2), `intermediate` (episodes via gaps-and-islands, trip assembly), `marts` (dimensions, facts, KPI marts) with `table_format='iceberg'`, incremental `merge` strategy and model contracts.
- **SCD**: dbt **snapshots** track vehicle and driver history (SCD2); dbt does not document Iceberg for snapshots, so snapshot tables are native Snowflake tables and the Iceberg `dim_vehicle` / `dim_driver` models are built from them with surrogate keys (`dbt_utils.generate_surrogate_key`). `dim_route` is SCD1 via incremental MERGE that overwrites.
- **Quality gate (write-audit-publish)**: `dbt build` runs tests right after each model (not_null, unique, relationships, accepted ranges via `dbt_expectations`, custom reconciliation tests). Only when the run is fully green does Airflow repoint the `SERVE` views to that run's timestamp (`AT(TIMESTAMP => …)`). A failed run leaves the API on the last good data.
- **Docs and lineage**: `dbt docs generate` gives the ERD/lineage evidence for the submission.

Pillar 5 · Orchestration & analytics (owner: C)

- Apache Airflow on Docker Compose (LocalExecutor). Providers: Airbyte (`AirbyteTriggerSyncOperator`), Snowflake (SQL operators), **astronomer-cosmos** to render the dbt project as an Airflow task group (one task per model + its tests).
- The dashboard is a FastAPI + React + three.js app (section 8). Every KPI chart reads a named view in `SERVE`, which makes reconciliation straightforward.

5 Dimensional model (gold, built by dbt)

Dimensions

Dimension	SCD	Key attributes (surrogate key `*_sk` on every dimension)
dim_vehicle	**Type 2**	vehicle_id, source_truck_id, make, model_year, reefer_capable, home_depot, reefer_unit_id, status; valid_from, valid_to, is_current
dim_driver	**Type 2**	driver_id, source_driver_id, name (Faker), license_class, home_terminal, assigned_vehicle_id, behaviour_profile
dim_route	**Type 1**	route_id, origin, destination, distance_km, speed_limit_kmh, route_name (corrections overwrite)
dim_facility / dim_customer	Type 1	From DS1; facility lat/lon + geofence radius
dim_cargo	static (seed)	cargo_type, is_cold_chain, setpoint_c, tolerance_c
dim_date / dim_time	static	Calendar (day grain) and time-of-day (minute grain)

SCD demo: `fleet.master.changes` reassigns ~2% of drivers and moves ~1% of vehicles between depots mid-run. The snapshot closes the old row and opens a new one; facts loaded afterwards point to the new surrogate key, while earlier facts keep the old one. Route name corrections overwrite in place (Type 1).

Facts and grain

Fact	Type	Grain (one row per…)	Main measures
fact_telemetry_ping	Transaction	vehicle reading at event_ts	speed, rpm, temps, fuel_rate, dist_km_since_prev, fuel_l_since_prev, flags
fact_vehicle_hourly	Periodic snapshot	vehicle × hour	km, litres, engine_on_min, idle_min, reefer_min, reefer_in_range_min
fact_trip	Accumulating snapshot	trip (load)	planned/actual km, litres, km/L, departure/arrival ts, idle_min, excursion_min
fact_idle_episode	Transaction (episode)	idle episode	start, end, duration_min, est_fuel_wasted_l
fact_coldchain_excursion	Transaction (episode)	excursion episode	duration_min, peak_temp_c, max_delta_c
fact_route_deviation	Transaction (episode)	deviation episode	duration_min, max_offset_km, extra_km
fact_driver_daily	Periodic snapshot	driver × day	km, harsh_events, speeding_events, idle_min, engine_on_min, safety_score
fact_fuel_purchase	Transaction (DS1)	fuel purchase	litres, cost (historical benchmark)

Metric additivity

Class	Measures	Rule
Additive	km, litres, idle_min, excursion_min, event counts	Sum across any dimension.
Semi-additive	fuel_level_pct, odometer_km, active vehicles	Sum across vehicles at one time; across time use last/avg value, never sum.
Non-additive	km/L, compliance %, idle %, safety score, averages	Never sum or average stored ratios; recompute from additive parts (Σkm / ΣL).

6 KPI definitions (documented, with units and thresholds)

KPI	Definition	Units / assumptions
Fuel efficiency per route	Σ distance_km ÷ Σ fuel_litres per route over the period. Distance = haversine between consecutive pings (gaps > 5 min excluded); fuel = fuel_rate_lph × Δt.	km/L (also L/100 km). Sanity-checked against DS1 fuel purchases.
Cold-chain compliance %	Reefer-active minutes within [setpoint − 2 °C, setpoint + 3 °C] ÷ reefer-active minutes, for cold-chain loads only. Secondary: % cold trips with zero excursions ≥ 15 min.	Percent. Adherence to the *simulation* threshold only, not regulatory certification. Sensor-gap minutes reported separately, not counted as compliant.
Idle %	Idle minutes (episode rule, section 3) ÷ engine-on minutes, per driver per day.	Percent; wasted fuel estimated at idle fuel rate.
Driver safety score	100 − (5 × harsh braking per 100 km + 3 × speeding events per 100 km + 0.5 × idle %), clipped to 0–100. Harsh: max_decel ≥ 3.5 m/s²; speeding: speed > route limit + 10 km/h for ≥ 2 pings.	Score 0–100; weights are documented assumptions, shown on the dashboard.
Pipeline health	Events/min ingested, DLQ and load-error rate, freshness lag, dbt test pass rate, detection precision/recall vs truth, credits per run.	Operational, for the demo and evidence.

7 Orchestration (Airflow DAGs)

DAG	Schedule	Tasks
`fleet_master_data`	daily / manual	airbyte_sync_kaggle → wait_snowpipe (COPY_HISTORY) → dbt snapshot → dbt build dims (Cosmos) → publish
`fleet_generator_bulk`	hourly	run_generator_bulk → airbyte_sync_bulk → wait_snowpipe → log_ingestion
`fleet_lakehouse_pipeline`	every 15 min	check_pipe_status → collect_load_errors → pyspark_silver (`snowpark-submit`) → dbt snapshot → dbt build staging → intermediate → marts (+ tests) → repoint_serve_views → detection_vs_truth → api_cache_invalidate → run_report
`fleet_stream_monitor`	5 min	Kafka Connect connector status (REST), consumer lag, DLQ rate < 1%, Snowpipe lag → alert / restart connector
`fleet_maintenance`	daily	credit usage report (metering views), clustering health, expired data checks, dbt docs generate
`fleet_dlq_replay`	manual	read DLQ + load errors → re-validate → resubmit → log outcome

All tasks are idempotent (incremental MERGE keyed by business key and run window), have `retries=3` with exponential backoff, and log run_id, row counts, durations and query IDs. Re-running a window must give identical gold results, and that is tested on Day 5.

8 Dashboard: FastAPI backend + React / three.js frontend

Backend (FastAPI)

- `snowflake-connector-python` with a dedicated XS `APP_WH` and read-only `APP_ROLE` on `SERVE` only.
- Endpoints: `/kpi/fuel-efficiency`, `/kpi/cold-chain`, `/kpi/drivers`, `/trips/{id}/temperature`, `/health/pipeline`; filters by date, depot, route, cargo type.
- Pydantic response models, OpenAPI docs; TTL cache (60 s) so the warehouse can auto-suspend; cache invalidated by Airflow after each publish.
- `/ws/positions`: an `aiokafka` consumer keeps the latest position per vehicle and pushes deltas at 1 Hz. This is live visualisation only; KPIs always come from gold.

Frontend (React + three.js)

- Vite + TypeScript, `@react-three/fiber` + `drei`, TanStack Query, Recharts (or ECharts) for 2D charts.
- 3D fleet map: lat/lon projected (Web Mercator) onto a plane; corridors as lines from the generator's corridor file; 5,000 vehicles as one **InstancedMesh** (one draw call) coloured by status; anomaly episodes as pulsing markers; click a vehicle for its detail panel.
- Pages: Live fleet (3D) · Route fuel efficiency · Cold-chain compliance · Driver safety & idling scorecard · Pipeline health (events/min, DLQ, freshness, dbt tests, detection precision/recall).

> **Scope guard:** the 3D map is one page and is timeboxed. If it slips, the four KPI pages (2D charts) still satisfy the handbook. Feature freeze for the UI is Day 5 evening.

9 Tools and alternatives

Layer	Chosen	Runs	Alternatives and when to switch
Generation	Python 3.11, asyncio, aiokafka, Faker, numpy	Local	threading / multiprocessing; Mimesis; SDV to fit synthetic tables
Data contract	JSON + JSON Schema (versioned in repo)	Local	Avro + Schema Registry (stronger evolution, +RAM); Protobuf
Streaming	Apache Kafka (KRaft) + Kafka Connect S3 sink + Kafka UI	Local	Redpanda (lighter, Kafka API); Confluent Cloud or Amazon MSK; Snowflake Kafka connector (Snowpipe Streaming) if bronze on S3 were not required

Layer	Chosen	Runs	Alternatives and when to switch
Batch ingestion	Airbyte OSS (demo machine) / PyAirbyte (dev laptops)	Local	Airbyte Cloud; Apache NiFi; Fivetran; plain Python extractors
Raw lake	Amazon S3 (versioning, Object Lock)	AWS	MinIO (local), ADLS, GCS
Raw load	Snowpipe auto-ingest (S3 events → SQS)	Snowflake	Scheduled `COPY INTO`; external tables; Airbyte S3 → Snowflake
Processing	PySpark on Snowpark Connect; local Spark for tests	Snowflake	Snowpark Python DataFrames (not PySpark, so weaker for Pillar 3); EMR / Databricks Spark
Transformation	dbt Core + dbt-snowflake, dbt_utils, dbt_expectations	Local CLI → Snowflake	dbt Cloud; Snowflake Dynamic Tables; SQLMesh
Table format	Apache Iceberg (Snowflake-managed, S3 external volume)	Snowflake	Native Snowflake tables (simpler, but no Iceberg for Pillar 4); Delta Lake
Warehouse / SQL	Snowflake (LOAD_WH, TRANSFORM_WH, APP_WH)	Snowflake	Trino / Athena over the same Iceberg files; BigQuery
Data quality	dbt tests + contracts + dbt_expectations	Snowflake	Great Expectations; Soda Core; Snowflake Data Metric Functions
Orchestration	Apache Airflow (Docker Compose) + astronomer-cosmos	Local	Amazon MWAA / Astronomer; Dagster; Prefect (Snowflake Tasks do not meet the Airflow requirement)
API	FastAPI, pydantic, snowflake-connector-python, aiokafka, uvicorn	Local	Flask; Node/Express; Redis for shared cache
Dashboard	React + Vite + TypeScript, three.js via react-three-fiber/drei, Recharts	Local	deck.gl / Mapbox GL for the map; ECharts; Superset or Metabase as a quick fallback
Infrastructure	Docker Compose, Makefile; Terraform (AWS + Snowflake providers)	Local	AWS CDK; schemachange for Snowflake DDL; plain SQL scripts
Engineering	GitHub PRs + review, pre-commit (ruff, black, sqlfluff, eslint), pytest + chispa, GitHub Actions	Local	GitLab CI

Local resource budget (16 GB laptops)

Profile	Services	Approx. RAM
`stream`	Kafka 1 GB, Kafka Connect 1 GB, Kafka UI 0.3 GB, generator 0.3 GB	~2.6 GB
`orchestrate`	Airflow scheduler + API/webserver + triggerer + Postgres (LocalExecutor), PyAirbyte in tasks	~2.5 GB
`app`	FastAPI 0.3 GB, React dev server 0.5 GB	~0.8 GB
`airbyte` (demo machine)	Airbyte OSS via `abctl` (needs several GB on its own)	~4–6 GB
`spark-local` (tests)	Spark driver + 1 worker on smoke profile	~3 GB

Heavy compute is in Snowflake, so laptops stay comfortable. Use the `airbyte` and `spark-local` profiles only when needed.

Snowflake set-up and cost guardrails

- One trial account for the team, created in the **same AWS region as the S3 buckets** (required for smooth Iceberg external volumes and no egress). Roles: `LOADER`, `TRANSFORMER`, `APP_ROLE`, `VALIDATOR`; databases `FLEET` with schemas RAW, SILVER, GOLD, SERVE, OPS.

- Warehouses all XS with `AUTO_SUSPEND=60`; scale `TRANSFORM_WH` to S only for the full benchmark. A resource monitor caps daily credits and suspends at 100%.
- AWS: S3 + SQS only (pennies). Budget alarm on the AWS account before creating any resource.

10 Team roles and ownership

Member A · Data & Ingestion

Pillars 1–2. Kaggle profiling and data contract; DS2 calibration; corridor builder; 5,000-vehicle generator, anomalies, DQ faults, truth log; Kafka + Kafka Connect S3 sink + DLQ; Airbyte connections; S3 bronze layout and Object Lock; Snowpipe + RAW + load errors; ingestion log; DLQ replay; detection-vs-truth harness; throughput benchmarks.

Member B · Snowflake, PySpark & dbt

Pillars 3–4. Snowflake roles, warehouses, resource monitor, external volume; PySpark silver job (Snowpark Connect + local); dbt project: staging, intermediate, marts, snapshots (SCD2), SCD1, episode detection (independent of A), KPI marts, tests, SERVE views; Iceberg features; performance and credit tuning.

Member C · Platform, API & Dashboard

Pillar 5 + app. Repo, CI, Docker Compose; Terraform for S3, IAM and SQS; Airflow DAGs with Airbyte, Snowflake and Cosmos; FastAPI backend; React + three.js dashboard; KPI reconciliation with B; README, evidence pack, demo recording; deck lead.

Responsibility matrix (R = responsible, A = accountable/approves, C = consulted)

Deliverable	A	B	C
Data profile, contract, calibration, generator	R/A	C	C
Kafka, Kafka Connect, Airbyte, S3 bronze, Snowpipe, DLQ, ingestion log	R/A	C	R (infra)
Snowflake account objects, roles, warehouses, external volume	C	R/A	C
PySpark silver job	C	R/A	
dbt model, SCD, facts, KPI marts, tests, SERVE views	C	R/A	C
Docker platform, Terraform (AWS), cost control		C	R/A
Airflow DAGs	C	C	R/A
FastAPI + React / three.js dashboard		C (SQL)	R/A
KPI reconciliation		R	R/A
Testing, evidence pack	R	R	A
README, architecture diagram, deck, demo video	R (own parts)	R (own parts)	R/A

Working agreement

- **Branching:** `main` protected; feature branches; every PR reviewed by one teammate (rotation A→B, B→C, C→A) against the review checklist (section 13).
- **Rhythm:** 15-min stand-up at start of day; **integration point at 17:00** daily, when everything merged so far runs end-to-end; end-of-day gate review.
- **Interfaces first:** the data contract, RAW table layout and the `SERVE` view columns are frozen on Day 1–2, so A, B and C can build in parallel. C builds the frontend against mocked API responses until the real views exist.

11 Six-day execution plan (mapped to the handbook's readiness gates)

Day	Member A	Member B	Member C	Exit gate (must be true to close the day)
Day 1 Scoping & foundations Gate 1	Download + profile DS1/DS2 (keys, nulls, units, ranges); data contract v1; city lookup + corridor builder; generator skeleton (50 vehicles)	Snowflake trial (same region), roles, warehouses, resource monitor, storage integration + external volume; ERD, grain, KPI sheet; dbt project skeleton	Repo, pre-commit, CI; Docker Compose (Kafka, Connect, Airflow, API, web); Terraform: S3 buckets, IAM role for Snowflake, SQS, budget alarm; React + FastAPI scaffolds	Scope, objectives and ownership agreed; architecture v1; contract + ERD reviewed; one hand-made event goes Kafka → S3
Day 2 Thin slice end-to-end Gate 2	Generator at 5,000 vehicles on corridors, trips from DS1 loads, seeded config; S3 sink + DLQ; Snowpipe → RAW; Airbyte Kaggle sync	PySpark silver v1 (parse, cast, dedupe) on Snowpark Connect and locally; dbt staging + initial dims from DS1	`fleet_master_data` DAG; FastAPI connected to Snowflake (one real endpoint); three.js scene with corridors	A generated event is traceable Kafka → S3 → RAW → SILVER → API; architecture v2
Day 3 Core build I Gate 3 (part)	3 anomaly families + DQ faults + sealed truth log; master-change events; Airbyte bulk sync; `OPS.INGESTION_LOG`; DLQ replay	Silver complete (null policy, quarantine, broadcast enrichment, corridor distance, lag windows, MERGE for late data); snapshots SCD2, SCD1 route; fact_telemetry_ping, fact_vehicle_hourly, fact_trip	`fleet_lakehouse_pipeline` with Cosmos task group + snowpark-submit; KPI endpoints on draft views; live map with WebSocket + 5,000 instanced vehicles	5,000 vehicles flowing continuously; SCD history visible; first facts queryable; live map moving
Day 4 Core build II Gate 3	Detection-vs-truth harness (precision, recall, delay); load-error evidence; schema v2 (`tire_pressure_kpa`); throughput benchmark v1	Episode facts (idle, excursion, deviation) via gaps-and-islands; fact_driver_daily + safety score; KPI marts; full test suite; SERVE views + repoint macro; clustering	All KPI pages + pipeline health page; full DAG run with publish and cache invalidation; run report	One orchestrated run goes raw → dashboard; all 3 KPIs visible from SERVE views
Day 5 Testing & refinement Gate 4	Failure tests: broker restart, malformed burst, S3 sink outage → retry, late events; full-profile benchmark	KPI reconciliation (dashboard = API = SQL = hand calc); time travel + failed-gate demo; query-profile pruning evidence; credit tuning; local vs Snowpark parity	DAG failure/retry and idempotent re-run tests; README run guide; fresh-clone reproduction on 2nd laptop; UI freeze in the evening	Every acceptance criterion demonstrable with saved evidence; defects fixed or logged as limitations
Day 6 Hardening & demo package Gate 5 prep	Data assets folder (schemas, configs, sample events, anomaly docs); deck: data & ingestion	Lakehouse evidence (table history, SCD, time travel, dbt docs + test results); deck: processing & model	Final architecture diagram; recorded fallback demo; deck assembly; contribution summary; cost report and teardown script	**Feature freeze 12:00.** Final `full` run; two demo rehearsals; readiness checklist fully ticked

> **Critical path:** data contract (D1) → generator + Snowpipe (D2) → silver (D3) → episodes + KPIs (D4) → reconciliation (D5). Anything not on this path, including the 3D map polish, can slip; anything on it is swarmed by the whole team at the 17:00 integration point if late.

12 Testing and validation plan

ID	Test	Pass criteria	Evidence / owner
T01	Generator coverage	5,000 distinct vehicles; all fields; all 3 anomaly families + DQ faults present	Count queries, truth-log summary · A
T02	Reproducibility of generator	Same seed + config → identical event hash for smoke profile	pytest · A
T03	Malformed burst	Every bad record in Kafka DLQ or OPS.LOAD_ERRORS with a reason; replay restores fixed ones	DLQ logs, replay report · A
T04	Broker / connector outage	Producer and Connect retry with backoff; counts reconcile after restart	Producer + Connect logs · A
T05	Duplicates and late events	0 duplicates in silver; late events merged into the correct day	dbt tests, MERGE results · B
T06	Count reconciliation	Kafka offsets = S3 records = RAW + load errors; RAW = silver + quarantine	Reconciliation model · A + B
T07	Quality gate blocks bad data	Injected bad batch fails a dbt test; SERVE views stay on the last good timestamp; Airflow task red	Airflow log, view DDL · B
T08	SCD behaviour	Reassigned driver has 2 rows; old facts keep old SK; Type 1 route overwritten	Before/after queries · B
T09	Time travel	Query a gold table `AT` an earlier timestamp and `BEFORE` a statement; results match the earlier run report	Query results · B
T10	Pruning	One-day filter scans a small fraction of partitions (query profile: partitions scanned vs total)	Query profile screenshot · B
T11	Detection accuracy	Recall ≥ 95%, precision ≥ 90% per family	Harness report · A
T12	KPI reconciliation	Dashboard = API = SQL = hand calc on a sampled vehicle/route/driver (±0.1%)	Reconciliation sheet · B + C
T13	Idempotent re-run	Re-running a DAG window yields identical gold row hashes	Airflow logs, hash query · C
T14	PySpark parity	Same job on local Spark and Snowpark Connect gives the same silver counts and hashes (smoke profile)	Parity report · B
T15	Fresh-clone reproduction	Second laptop runs `make up && make demo` in ≤ 30 min	Screen recording · C
T16	Performance and cost (measured)	Events/s, load latency, job runtimes, credits per run recorded	benchmark.csv, metering views · A + B
T17	Dashboard behaviour	3D map renders 5,000 vehicles at ≥ 30 fps on the demo laptop; API p95 < 500 ms with warm cache	Browser perf capture, API log · C

13 Repository, conventions and senior review

```
fleet-lakehouse/
├─ Makefile  .env.example  README.md
├─ docker/            compose.yaml (profiles)
├─ infra/terraform/   aws/ (s3, iam, sqs, budget)
│                     snowflake/ (roles, wh, stages,
│                     pipes, external volume)
├─ contracts/         telemetry.v1.json  v2  changes
├─ generator/         fleet_sim/ (shards, corridors,
│                     anomalies, sinks)  generator.yaml
├─ ingestion/         kafka-connect/ airbyte/ snowpipe/
│                     dlq_replay.py
├─ spark_jobs/        silver_telemetry.py (+ tests)
├─ dbt/fleet/         models/ staging intermediate marts
│                     snapshots/ seeds/ tests/ macros/
├─ airflow/dags/      fleet_*.py
├─ api/               FastAPI app (routers, schemas)
├─ web/               React + three.js (Vite, TS)
├─ tests/             unit/ spark/ e2e/
├─ data_samples/      sample events, profiles
└─ evidence/          dbt docs, logs, screenshots, bench
```

Definition of done (every PR)

- Runs locally and against the shared Snowflake dev schema (per-member dbt target).
- Idempotent; no hard-coded paths or secrets (use `.env`, key-pair auth for Snowflake).
- Unit, Spark or dbt test added; smoke profile passes in CI.
- Schema or contract changes are versioned and documented.
- Row counts, timings and query IDs are logged; failures are visible, never swallowed.
- KPI changes update the definition table, the dbt test and the reconciliation SQL.
- Evidence (screenshot/log) saved to `evidence/` when it proves a requirement.

Senior review checkpoints

- End of D1: contract, ERD, grain, KPI formulas (no build before sign-off).
- End of D2: thin slice raw → API.
- End of D4: full run + dashboards.
- End of D5: test plan results, evidence pack.

14 Risks and mitigations

Risk	Impact	Mitigation
Snowpark Connect limits or unsupported PySpark API in our job	High	Check its compatibility list on Day 1 and keep the job to standard DataFrame APIs. Fallback: run the same job on local Spark reading S3, write Snappy Parquet to `s3://lakehouse/silver_stage/`, and load it with Snowpipe. Pillar 3 is still met.
dbt snapshots not supported as Iceberg	Med	Snapshots as native tables, Iceberg dims built from them (already in the design).
Kaggle schemas differ from expectations	High	Day 1 profiling; renames handled in dbt staging; contract frozen only after profiling.
Snowflake credit burn	Med	XS warehouses, 60 s auto-suspend, resource monitor, API cache, 15-min schedule instead of continuous.
Airbyte OSS RAM on laptops	Med	PyAirbyte on dev laptops; Airbyte OSS only on the demo machine.
Three.js scope creep	Med	Mocked API from Day 1; 3D map timeboxed; KPI pages are 2D and come first.
Region mismatch S3 vs Snowflake	Med	Create the Snowflake account in the S3 region on Day 1, before anything else.
Detection leaks labels	Med	Truth prefix excluded from the storage integration; A/B independence rule.
Streaming instability during demo	Med	Recorded fallback video; pre-loaded `full` run so dashboards are populated before going live.
Integration surprises at the end	High	Daily 17:00 end-to-end integration from Day 2 onward.

15 Submission package mapping

Handbook deliverable	What we hand in	Owner
01 Source code	Repository above: generator, ingestion configs, PySpark, dbt, Airflow DAGs, API, web app, Terraform, compose	All
02 Technical documentation	README (setup for local + AWS + Snowflake, dependencies, configuration, run steps), architecture diagram, dbt docs/ERD, KPI definitions	C
03 Data assets	Generator, calibration, contracts v1/v2, anomaly specs, sample events (incl. DLQ samples), Kaggle profiling report	A
04 Technical evidence	dbt test results, ingestion/DLQ/load-error logs, PySpark outputs, Iceberg table history and time travel, SCD before/after, Airflow logs, detection report, benchmarks, credit report	A + B
05 Working solution	Executable pipeline, Iceberg lakehouse in Snowflake, star schema, SERVE views, FastAPI + React dashboard	All
06 Presentation deck	Problem, architecture, implementation, results, limitations, business value	C (lead)
07 Demonstration	Live demo script (≤ 12 min) + recorded fallback	C
08 Contribution summary	Per member: components built, evidence produced, PRs reviewed	All

Requirement traceability (Project 03 acceptance criteria)

Acceptance criterion	Where it is satisfied	Test
Configuration and output evidence cover 5,000 distinct vehicles	generator.yaml; dim_vehicle; silver distinct count	T01
All telemetry fields and three anomaly families present	Contract v1; truth log; episode facts	T01, T11
Route, cold-chain and driver KPIs reconcile with analytical queries	KPI marts; SERVE views; API; dashboard	T12
A complete orchestrated run connects raw events to warehouse and dashboard	`fleet_lakehouse_pipeline` + run report	T13, T15
Shared: immutable raw, DLQ, retry, logs	Pillar 2 design	T03, T04, T06
Shared: PySpark processing	Pillar 3 design	T05, T14
Shared: ACID, time travel, MERGE, pruning, SCD1/2, star schema, quality gates	Pillar 4 design + section 5	T07–T10
Recommended: units/thresholds stated; throughput and resources recorded	KPI table (section 6); benchmark.csv; credit report	T16

16 Open items to confirm

- Member names, to replace A / B / C in the deck and contribution summary.
- Exact DS1/DS2 schemas (Day 1 profiling): file names, units, ID formats.
- Snowflake trial region (must match the S3 region) and who owns the account.
- Snowpark Connect availability on the trial account (Day 1 smoke test), so the fallback decision is made early.
- Demo date and slot length, so the demo script and recording are sized correctly.